Automated Investment System

System Outline

The system is designed to allow users to deposit funds into an account, which are then automatically invested according to predefined strategies based on the user's risk profile. It is built to be secure, reliable, and scalable, while adhering to South African financial regulations.

Requirements

The functional and non-functional requirement of the system as outlined in the specification brief have been tabulated below.

Functional Requirements

User Onboarding	Allow new users to create an account.
Deposit Funds	Users must be able to deposit money into their account through various methods (e.g., bank transfer, card payment).
Investment Strategy	The system needs a mechanism to determine the correct assets for investment. This could be based on user risk profiles, pre-defined portfolios, or other criteria.
Automated Investment	Once funds are deposited, the system should automatically execute trades to invest the money according to the determined strategy.
Account Viewing	Users should be able to view their account balance, holdings, and transaction history.

Non-functional Requirements

Reliability	The system must reliably process deposits and execute investments. Financial transactions must be accurate and atomic.
Scalability	The system should handle a growing number of users and transactions.
Security	Protect user data, financial information, and prevent unauthorized access. Ensure secure handling of transactions.

Performance	Deposits and investment executions should occur within a reasonable timeframe. Users should experience low latency when viewing their account information.
Compliance	The system must adhere to relevant financial regulations and reporting requirements.

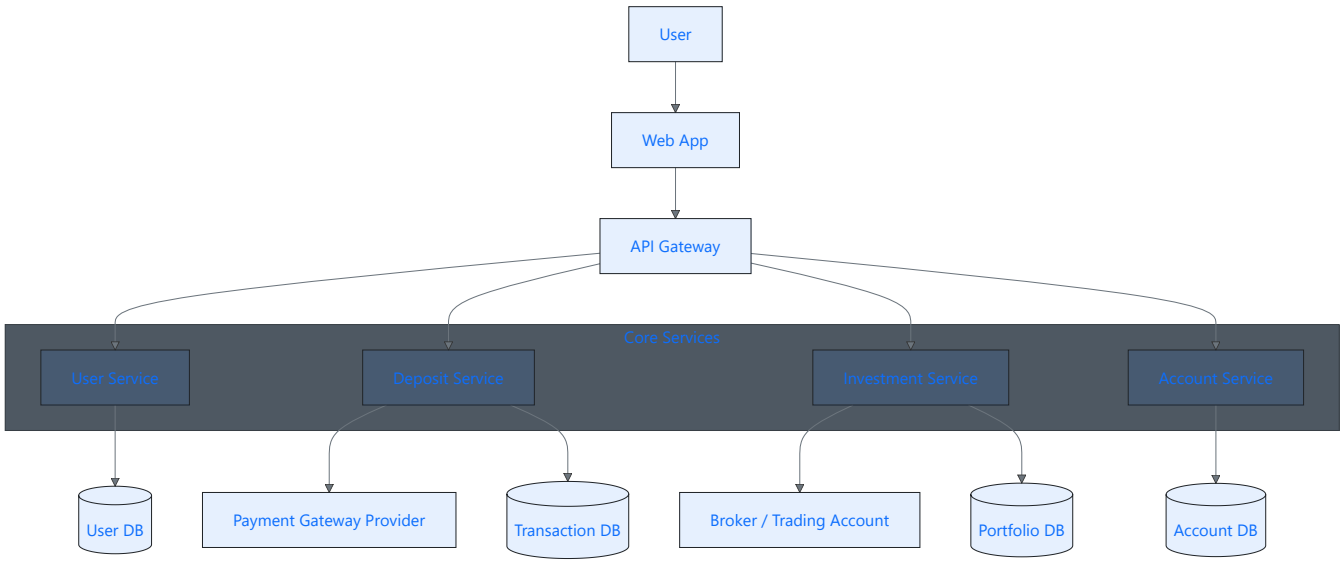
System Architecture

The overall high-level architecture consists of a web application, which serves as the interface for users to interact with the system, and an API Gateway, which routes requests from the web app to the system's core services.

The backend is composed of four main core services:

1. User Service
2. Deposit Service
3. Investment Service
4. Account Service

These services interact with various databases to persist data and with external integrations. This architecture ensures secure, reliable, and scalable operation, separating concerns between the frontend, backend services, and external systems, while providing a clear, maintainable structure for the automated investment system.



Key components

The following section proivdes a detailed explanation of the key components illustrated in the architecture diagram.

Web Application

The Web Application is the primary interface through which users interact with the investment system. Its main responsibilities include:

- Allows users to sign up, complete KYC, deposit funds, view account balances, and track portfolio performance.
- Provides dashboards for transaction history, portfolio allocation, and real-time updates on holdings.

API Gateway

The API Gateway acts as the central entry point for all client requests and orchestrates communication with the system's backend services:

Request Routing:

- Routes incoming API requests from the Web Application to the appropriate core service (User, Deposit, Investment, Account).

Authentication & Authorization:

- Validates JWT tokens and ensures that only authenticated and authorized users can access protected endpoints.

Request Management:

- Performs input validation, rate limiting, and throttling to prevent abuse and maintain service stability.
- Provides centralized logging and monitoring of all API requests, enabling performance tracking and troubleshooting.

Core Services

a) User Service

The User Service is responsible for managing all aspects of user accounts, from registration to authentication and compliance:

User Onboarding:

- Handles sign-up, registration, and profile creation via the Web App.
- Collects necessary user information such as name, email, contact details, and preferences.

KYC (Know Your Customer) Checks:

- Performs verification to comply with FICA and other financial regulations.

- Validates documents, identity, and potentially risk assessment forms.

Authentication & Session Management:

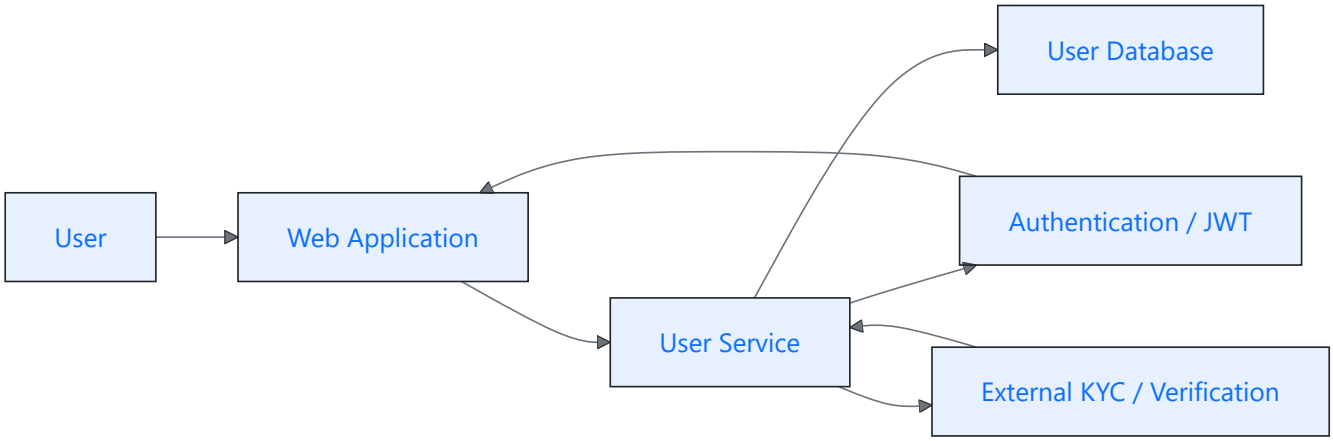
- Authenticates returning users using email/password or other credentials.
- Issues JWT tokens to manage secure sessions and authorize API requests.

User Data Management:

- Stores user profiles, preferences, KYC status, and authentication info in UserDB.
- Provides APIs for profile updates, password changes, and retrieval of user information.

Security & Compliance:

- Ensures encrypted storage of sensitive data (passwords, documents).
- Implements access control so only authorized services can read/write user data.



---

b) Deposit Service

The Deposit Service is responsible for securely accepting and recording funds from users into their accounts. Its responsibilities include:

Initiate Deposit:

- Receives a deposit request from the Web App when a user chooses a payment method (bank transfer, card payment, or other supported channels).

Payment Gateway Integration:

- Connects to a trusted external payment gateway (e.g., PayGate, Ozow, or local bank APIs) to process the transaction.
- Handles various payment types, including EFTs, debit/credit cards, and instant payment systems.

Transaction Verification & Security:

- Ensures all deposits are authenticated, authorized, and encrypted

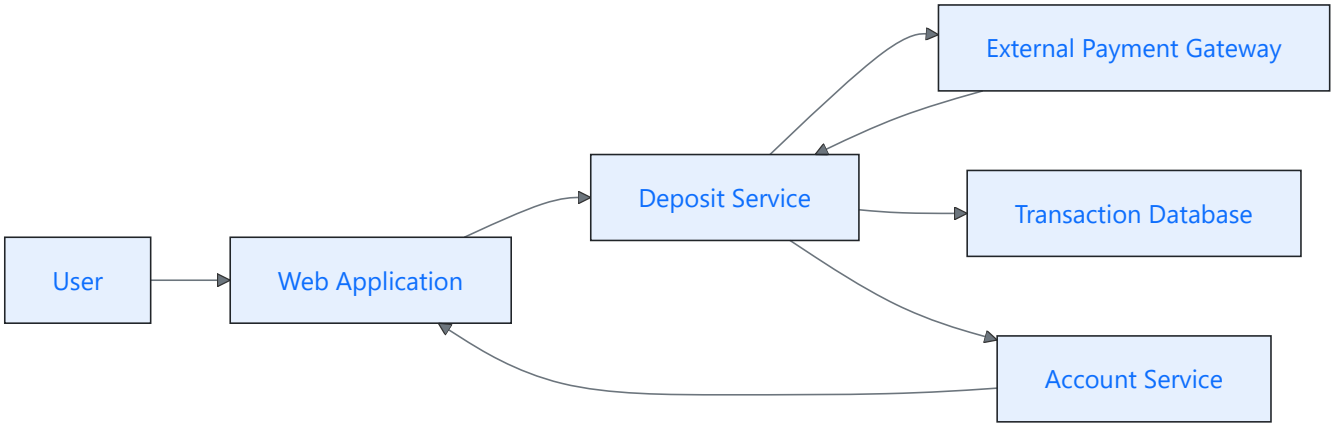
- Validates the payment confirmation from the gateway before updating internal records.

Record Deposit:

- Updates the TransactionDB with details such as amount, timestamp, payment method, and status.
- Ensures atomicity, so either the deposit is fully recorded or no partial updates occur.

Notify Other Services:

- Sends a message or event to the Account Service to update the user's account balance.
- Optionally triggers the Investment Service if funds are to be automatically invested.



---

c) Investment Service

The Investment Service is responsible for turning user deposits into actual investments and keeping portfolios up-to-date. Its workflow includes:

Risk Profiling:

- Determines the user's risk tolerance based on preferences, questionnaires, or pre-defined profiles.
- Assigns a risk category (e.g., conservative, balanced, aggressive).

Strategy Selection:

- Maps the user's risk profile to a suitable investment strategy or portfolio allocation.
- Can use predefined portfolios (e.g., ETF allocations) or algorithmic rules

Trade Execution:

- Sends orders to an external broker API (e.g., SatrixNow) to invest funds.
- Ensures atomic execution: trades either complete fully or fail safely, avoiding mismatched allocations.

Portfolio Updates:

- Updates the PortfolioDB to reflect holdings, allocations, and asset values.

- Tracks transaction history for reporting, performance calculation, and account viewing.

Monitoring & Error Handling:

- Monitors trade confirmations from the broker.
- Retries failed trades or flags issues for manual review

---

d) Account Service

The Account Service is responsible for maintaining an up-to-date view of each user's financial information and ensuring that all account data is accurate and accessible:

Account Balances:

- Tracks the current cash balance for each user, reflecting deposits, withdrawals, and investment activities.
- Handles real-time updates when funds are added, invested, or withdrawn

Transaction History:

- Records all financial transactions, including deposits, withdrawals, trades, fees, and dividends.
- Supports filtering, sorting, and querying to allow users to review their activity over time.

Portfolio Holdings:

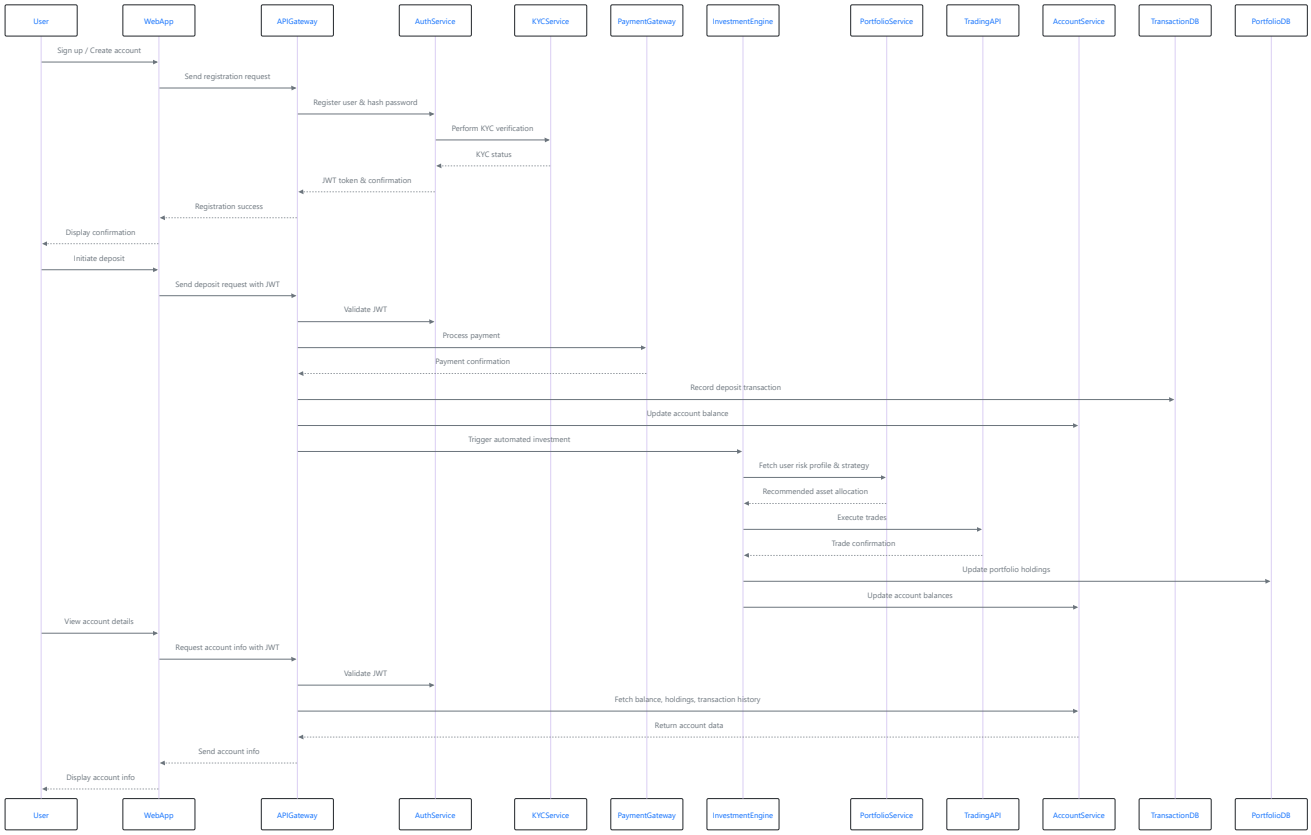
- Maintains a record of all investments held by the user, including quantity, value, purchase date, and allocation percentages.
- Supports performance calculations, such as ROI, gains/losses, and risk metrics.

Data Aggregation & Presentation:

- Aggregates data from the Deposit Service, Investment Service, and TransactionDB to provide a cohesive view of the user's financial status.
- Formats account data in a user-friendly way for the web app, ensuring fast retrieval and minimal latency.

Sequence Diagram

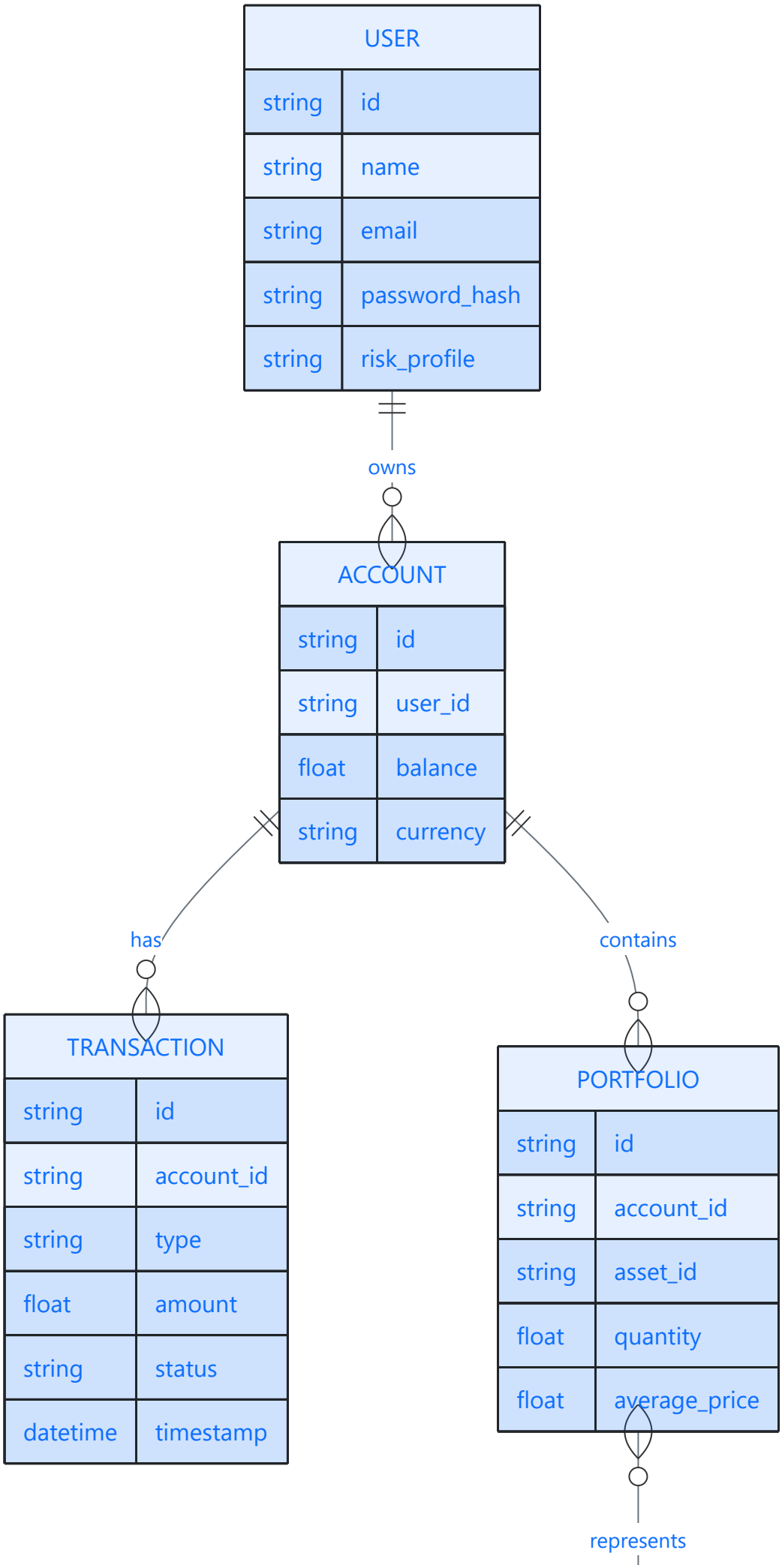
The following sequence diagram illustrates the flow of interactions between the user, the web application, the API gateway, core services, external integrations, and databases within the system.

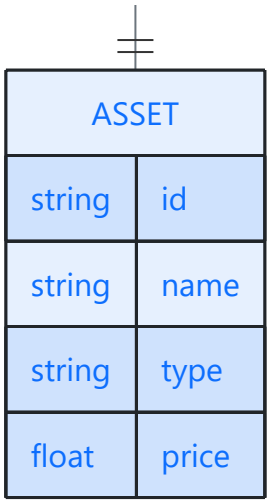


Data models

While not exhaustive, this section outlines the core data models that form the foundation of the system. These entities define how user, account, transaction, and investment data are structured, stored, and related within the system's databases.

Entity	Key Attributes	Relationships
User	id, name, email, password_hash, risk_profile	1 user → 1 account
Account	id, user_id, balance, currency	1 account → many transactions & portfolio items
Transaction	id, account_id, type (deposit/investment), amount, status, timestamp	Belongs to an account
Portfolio	id, account_id, asset_id, quantity, average_price	1 account → many assets
Asset	id, name, type, price	Linked to portfolios


USER
string id
string name
string email
string password_hash
string risk_profile
owns
ACCOUNT
string id
string user_id
float balance
string currency
has
contains
TRANSACTION
string id
string account_id
string type
float amount
string status
datetime timestamp
PORTFOLIO
string id
string account_id
string asset_id
float quantity
float average_price
represents



Technology justification

Frontend

The frontend will be a web application that allows users to interact with their accounts, view holdings, and monitor investments.

We will use React (JavaScript) to build the interface.

Justification:

- `Component reusability`: Enables modular development and easier maintenance.
- `Fast rendering`: Optimized updates via the virtual DOM, providing a smooth user experience.
- `Rich ecosystem`: Extensive libraries, developer tools, and community support facilitate rapid development and future enhancements.

Backend

Category	Technology	Justification
Programming Language	**Python 3.10+**	Mature, readable, and widely used in fintech and data-driven applications. Offers strong ecosystem support for APIs, financial computation, and async operations.
Web/API Framework	**FastAPI + Uvicorn + Pydantic**	High-performance, asynchronous framework ideal for scalable APIs. Provides automatic request validation,

Category	Technology	Justification
		OpenAPI documentation, and type safety through Pydantic models. Its async nature ensures efficient handling of concurrent I/O tasks like payments or trade executions.
Database	**PostgreSQL**	Reliable, ACID-compliant relational database suited for financial applications. Supports strong consistency, transactional integrity, and advanced features like JSONB for flexible data modeling.
ORM / DB Layer	**SQLAlchemy**	Integrates smoothly with FastAPI. Simplifies database interactions, migrations, and schema definitions with robust ORM capabilities.
Authentication / Security	**JWT (JSON Web Tokens)**	Enables stateless, secure authentication between frontend and backend. Ensures protected access to user data and account actions.

External integrations

Payment gateway

This is an external, third-party that would handle the user funds deposits into the system, and would allow users to deposit funds via bank transfer and card / virtual card payments.

Potential providers:

- PayFast
- Local banking payment portals
- Ozow

These providers have been selected because they comply with South African financial regulations and operate under FIC Act (FICA) and FSCA oversight. Their adherence to local KYC (Know Your Customer) and AML (Anti-Money Laundering) standards ensures that all financial transactions are secure, traceable, and compliant with national regulatory frameworks.

Broker / investment platform API

This is the component where all investments and trades are executed. A suitable provider for the South African market would be SatrixNow.

Justification:

- `Accessible`: No minimum investment requirements, making it suitable for all users.
- `Cost-effective`: Low fees for transactions and fund management.
- `Diverse offerings`: Provides both local and global ETFs and unit trusts.
- `Regulated`: Fully South African, adhering to FCSA, FICA, and other local financial regulations.

Market data

The system integrates with third-party market data providers to obtain accurate and timely financial information. This data informs investment decisions, portfolio valuations, and performance tracking.

Examples of data sources:

- `JSE Market Data Feeds`: Provides real-time and end-of-day prices for South African equities, ETFs, and indices.
- `Yahoo Finance API`: Offers historical and current prices for global and local assets, useful for portfolio valuation and international investments.

Addressing Non-functional Requirements

Security

- `Authentication & Access Control:` Strong user authentication with hashed passwords and two-factor verification (2FA) ensures only authorized access.
- `JWT Authorization`: JSON Web Tokens (JWTs) manage secure, stateless sessions between the frontend and backend. Tokens are time-limited and signed.
- `Data Protection`: All communication occurs over HTTPS (TLS 1.2+), and sensitive data is encrypted at rest (e.g., using AES-256).

Reliability

- `Atomic transactions`: Deposits, withdrawals, and investment executions are performed as atomic operations, ensuring either complete success or full rollback on failure.
- `Error handling & retries`: Failures in external services (payment gateways, broker APIs) are retried automatically, with alerts for manual intervention if needed.
- `Monitoring and alerts`: Real-time monitoring tracks system health, and administrators are notified of failed operations or unusual activity.

- `Redundancy & backups`: Critical databases and services are replicated and backed up to ensure minimal downtime in case of hardware or software failures.

Scalability

The system is designed to handle a growing number of users and transactions efficiently:

- `Stateless services`: Core services (User, Deposit, Investment, Account) are stateless, enabling horizontal scaling with additional instances as demand grows.
- `Asynchronous processing`: Investment executions, external API calls, and notifications are handled asynchronously to avoid blocking and improve throughput.
- `Connection pooling and caching`: Database connections are pooled, and frequently accessed data (e.g., market data, user portfolios) can be cached using Redis or a similar in-memory store.
- `Rate limiting and throttlin`g: Protects APIs from abuse and ensures fair resource usage during traffic spikes.

Regulation

The automated investment system is designed with regulatory compliance in mind. Key approaches include:

`Leveraging Regulated Third Parties`

- All financial operations (payments, trades, deposits, withdrawals) are routed through payment gateways and investment APIs that are already licensed and regulated in their jurisdictions.
- This ensures that core financial operations meet existing regulatory standards without building custom compliance from scratch.

`User Identity Verification (KYC)`

- The system integrates with external KYC/AML services to verify user identity before allowing deposits or investments.
- This ensures adherence to anti-money laundering regulations and prevents fraudulent activity.

`Data Privacy and Security`

- All personal and financial data is encrypted in transit (TLS/HTTPS) and at rest.
- The system follows data protection regulations such as GDPR or local privacy laws by limiting data retention, giving users control over their personal information, and anonymizing sensitive analytics where possible.

`Auditability and Reporting`

- All investment transactions are logged with timestamps and metadata for audit purposes.
- In case of regulatory requests, the system can produce accurate transaction histories and compliance reports.

Potential Failure Points & Handling

In any automated investment system, certain operational risks can disrupt services or compromise data integrity. Identifying these potential failure points and implementing appropriate mitigation strategies is critical to ensure reliability, security, and a smooth user experience. Below are three key risks and how they can be managed:

Database Failure

- `Issue`: A database outage could prevent access to critical user, account, or portfolio data.
- `Mitigation`: Redundancy and backups were implemented, including replicated databases and regular backups, to ensure minimal downtime and prevent data loss. This is why atomic transactions and error handling are also employed — so operations either complete fully or roll back safely, preserving data integrity.

Network / Service Latency

- `Issue`: Slow responses from the network or dependent services (e.g., payment gateways, broker APIs) can delay deposits, trade execution, or account updates.

- `Mitigation`: Asynchronous processing and connection pooling allow the system to handle requests without blocking, improving throughput. Additionally, rate limiting and retries ensure reliable communication with external services, which is why monitoring and alerts were implemented — to detect and respond quickly to service delays.

Data Leaks / Security Breaches

- `Issue`: Sensitive user information, financial data, or authentication credentials could be exposed through cyber attacks or misconfigurations.

- `Mitigation`: Strong authentication and 2FA, JWT authorization, and encryption at rest and in transit protect user data. This is why HTTPS (TLS 1.2+) and AES-256 encryption are used, ensuring that financial and personal data remain secure even if a breach is attempted.